

Correctness of the bit-blasted lower-rounding circuit

A pen-and-paper proof, structured for mechanisation in Lean 4

Target file: Fp/Theorems/UnpackedFloat/Round.lean — theorem UnpackedFloat.blastLower_Rel_smtLibLower

0.1 Setup

We round into two formats: `PackedFloat e s`, and `PackedFloat e (s+1)`, the same format with one extra significand bit. `lower r` is the greatest representable value below r , and `upper r` the least above (this is `IsLawfulLower / IsLawfulUpper`). Write a for the embed of the coarse `lower r`, and δ for the grid spacing at r , so r lives in $[a, a + \delta)$.

Three facts we use throughout:

- The fine grid is the coarse grid plus the midpoint of every cell. So the only fine points in $[a, a + \delta)$ are a and $a + \delta/2$.
- The guard bit g says which half of the cell r is in: $g = 0$ iff $r < a + \delta/2$. The sticky bit t is 0 iff r is on the fine grid, i.e. r is a or the midpoint.
- For negative r , guard and sticky refer to $|r|$ (neg only flips the sign bit, so the circuit reads the same significand).

0.2 Correctness of lower (also upper)

Check that `lower` produces an actual floating point number. This needs us to show that the result of `lower` can be packed: `(lowerComputed r).pack.unpack = lowerComputed r` under the conditions we call it.

Next, we show that `lowerComputed r` is close enough to r . That is, (a) `lowerComputed r <= r`, and (b) $r - \delta < \text{lowerComputed } r$ (concretely, $\delta = 2^{-(s+2)}$).

From this, `lowerComputed r = lower r`. First, `lowerComputed r <= lower r`, since `lowerComputed r` is a lower bound on r and must be dominated by the greatest lower bound. Next, assume for contradiction that `lowerComputed r  $\neq$  lower r`.

Then `lowerComputed r + delta <= lower r <= r`, by the discreteness lemma for `PackedFloat`. This contradicts (b). This proves `lower` correct.

0.3 Correctness of upper

Same strategy, mirrored: show $r <= \text{upperComputed } r < r + \delta$, then run the least-upper-bound argument. The circuit is different (it increments the significand, with carry), but the proof only consumes the two bounds and discreteness.

0.4 The half-ulp identity

`isLowerHalf` and `isTieBreak` compare rounding in the coarse format against the fine one. Everything follows from one identity. For $r \geq 0$:

$$\text{fine lower} = a + g \cdot \delta/2. \tag{1}$$

That is: the fine lower is the coarse lower, bumped up by half an ulp exactly when the guard bit is set. Proof: a is fine-representable and below r , so $a \leq \text{fine lower} \leq r < a + \delta$. The only fine points in that range are a and the midpoint, and which one we land on is decided by which half r is in — which is g .

Dually for upper: if $t = 0$ then `fine upper = fine lower = r`; if $t = 1$ then the fine upper is the midpoint when $g = 0$, and $a + \delta$ when $g = 1$.

0.5 Correctness of `isLowerHalf`

In SMT-LIB:

```
lowerHalf r := smtLibEq (v.embed (v.lower r)) (ves.embed (ves.lower r))
```

i.e. refining the grid does not change the result of rounding down. (`smtLibEq` is plain equality away from NaN and signed zeros.)

For $r \geq 0$: by the half-ulp identity, the two lowers agree iff $g = 0$. So `lowerHalf` = `!guardBit`, which is exactly `blastIsLowerHalfNonneg`.

For $r < 0$: negation swaps lower and upper. So `lowerHalf r` iff the coarse and fine uppers of $|r|$ agree, which (for $t = 1$, by the dual identity) happens iff $g = 1$. So `lowerHalf` = `guardBit` of the magnitude, which is exactly `blastIsLowerHalfNeg`.

Boundary cases ($t = 0$, negative side only): at an exact point the spec says true but the circuit says false; at a midpoint the spec says false but the circuit says true. Both are harmless. An exact r has `lower` = `upper` = r , so every mode returns r regardless. A midpoint is a tie, and the tie branches fire before `lowerHalf` matters. So we mechanise the pointwise lemma under $t = 1$, and discharge $t = 0$ inside the round proof.

0.6 Correctness of `isTieBreak`

```
tieBreak r :=
  (v.embed (v.lower r) < ves.embed (ves.lower r)) =
  (ves.embed (ves.upper r) < (v.embed (v.upper r)))
```

i.e. refining raises the lower exactly when it lowers the upper. Case on (g, t) , using the identities:

- $(0, 0)$, exact: both sides false. True.
- $(0, 1)$: lowers agree, uppers do not. False.
- $(1, 0)$, midpoint: both sides true. True — the genuine tie.
- $(1, 1)$: lowers differ, uppers agree. False.

So the spec holds exactly when $t = 0$: midpoints *and* exact points. The circuit computes `guard && !sticky` — midpoints only. The mismatch at exact points is harmless as before, so we mechanise under “ r not representable in the coarse format”.

Negation swaps the two sides of the biconditional, which is symmetric, so the same boolean works for both signs — as the circuit does.

Caveat: near overflow the coarse upper goes infinite while the fine format still has finite points above `maxNormal`. So the lemma needs an in-range hypothesis; the circuit handles overflow on a separate path anyway.

0.7 Correctness of `isEvenLower`

```
isEven := roundableIsEven_of_packedFloat.isEven
```

`isEven` reads the LSB of the packed significand. Once `lower` is correct, both sides look at the same float, and we just check that `blastExtractIsEven` (bit `guardBitIndex + 1` of the wide significand) is that LSB — a `getLsbD` index computation, transported through `pack/unpack`.

0.8 Correctness of `isEvenUpper`

The circuit returns `!isEvenLower`. This is right because adjacent floats have significands of opposite parity: when r is not exact, `upper r` = `succ`

(`lower r`) (nothing representable lies in between), and `succ` increments the significand by one — which always flips the LSB, including the carry case (all-ones, odd, to zero, even) and across binade and subnormal boundaries. For this we need a `PackedFloat.succ` with these properties.

When r is exact the claim is false pointwise but never used: parities are only consulted at ties. When $\mathbf{upper}$ is infinite, the overflow path applies.